

LLMProvider

LLMProvider

एक इंटरफ़ेस, ४३ प्रदाता — सर्किट ब्रेकर, पुनःप्रयास और स्वास्थ्य निगरानी के साथ।

सारांश

LLMProvider एक सामान्य, पुनःप्रयोगी Go मॉड्यूल है जो एकीकृत LLMProvider इंटरफ़ेस को परिभाषित करता है, साथ ही इसके आसपास उत्पादन-स्तरीय लचीलापन पैटर्न — सर्किट ब्रेकर, स्वास्थ्य निगरानी, बैकऑफ़ के साथ पुनःप्रयास, आलसी लोडिंग — और इस एक अनुबंध के पीछे ४३ ठोस प्रदाता कार्यान्वयन प्रदान करता है। इसमें OpenAI-संगत सामान्य एडेप्टर और ईमानदार, बिना-हार्डकोडेड-फॉलबैक मॉडल खोज शामिल है।

संक्षिप्त विवरण

एक पुनःप्रयोगी Go मॉड्यूल जो एक LLMProvider इंटरफ़ेस (Complete, CompleteStream, HealthCheck, GetCapabilities, ValidateConfig) को उजागर करता है, साथ ही दोष-सहनशीलता तंत्र — सर्किट ब्रेकर, स्वास्थ्य मॉनिटर, ज़िटर्ड बैकऑफ़ पुनःप्रयास, आलसी आरंभ — ४३ प्रदाता एडेप्टर और एक सामान्य OpenAI-संगत एडेप्टर के साथ। थ्रेड-सुरक्षित।

विस्तृत विवरण

LLMProvider वह अमूर्त परत है जिसकी हर LLM-उपभोग करने वाली सेवा को आवश्यकता होती है, लेकिन जिसे लगभग कोई भी ठीक से नहीं बनाता — वह अनाकर्षक आधारभूत संरचना जो एक डेमो को उस प्रणाली से अलग करती है जो वास्तविक ट्रैफ़िक के संपर्क में आने पर भी जीवित रहती है। यह एकल, क्षमता-सचेत इंटरफ़ेस — Complete, CompleteStream, HealthCheck, GetCapabilities, ValidateConfig — को परिभाषित करता है, ताकि एप्लिकेशन कोड केवल एक अनुबंध को लक्षित करे, चाहे ४३ बैकएंड में से कोई भी कॉल का उत्तर दे। फिर यह परिचालन सुदृढ़ीकरण प्रदान करता है जो नाजुक प्रदाता कॉलों को उत्पादन में चलाने योग्य बनाता है, बिना सांस रोके।

एक तीन-स्थिति सर्किट ब्रेकर (बंद → खुला → अर्ध-खुला) किसी भी प्रदाता को पारदर्शी रूप से लपेटता है — इसके स्ट्रीमिंग चैनल सहित, जहाँ एक खाली स्ट्रीम को सही ढंग से विफलता के रूप में गिना जाता है — ताकि एक खराब बैकएंड खुलकर पूरे सेवा को गिराने से रोक सके; एक केंद्रीय CircuitBreakerManager हर ब्रेकर को एक साथ ट्रैक करता है। एक

विन्यास योग्य स्वास्थ्य मॉनिटर प्रदाताओं को निरंतर स्वस्थ / अधोगामी / अस्वस्थ / अज्ञात स्थितियों में जाँचता है, ताकि गिरावट का अवलोकन किया जा सके, न कि आउटेज के माध्यम से इसकी खोज।

पुनःप्रयास तर्क में घातीय बैकऑफ़ के साथ ज़िटर जोड़ा गया है, जो स्थिति-सचेत निर्णय लेता है — उन त्रुटियों को पुनःप्रयास करें जिनका पुनःप्रयास सार्थक है (429, 5xx, क्षणिक नेटवर्क दोष), 4xx या रद्द किए गए संदर्भ पर चक्र बर्बाद न करें — विलंब को सीमित किया गया है ताकि बैकऑफ़ तूफान नियंत्रण से बाहर न हो। और आलसी-आरंभ पैटर्न प्रत्येक प्रदाता के निर्माण को उसके पहले वास्तविक उपयोग तक टालता है — एक जानबूझकर किया गया डिज़ाइन निर्णय जो सभी ४३ प्रदाताओं के पंजीकरण को लगभग निःशुल्क बनाए रखता है।

यह मॉड्यूल ४३ ठोस प्रदाता पैकेज के साथ-साथ एक generic OpenAI-संगत एंडेप्टर भी प्रदान करता है, जो पूर्ण इंटरफ़ेस को किसी भी /v1/chat/completions एंडपॉइंट के विरुद्ध लागू करता है — Bearer प्रमाणीकरण, SSE स्ट्रीमिंग के साथ सही [DONE] हैंडलिंग — ताकि बिना समर्पित पैकेज वाले विक्रेता को भी आप एंडेप्टर को उसके URL की ओर इंगित करते ही प्रथम श्रेणी का नागरिक बना सकें।

प्रमाण-पत्र एक ही स्थान (apikeys, ApiKey <Provider> के सख्त परंपरा का उपयोग करते हुए) पर हल किए जाते हैं, जिससे “हार्डकोडेड कुंजी परीक्षण सूट पास करती है, वास्तविक कुंजी कभी जुड़ी नहीं थी, उत्पाद उत्पादन में विफल हो जाता है” जैसी पूरी श्रेणी की बग का स्रोत पर ही समाधान हो जाता है। मॉडल खोज जानबूझकर, लगभग हठपूर्वक ईमानदार है: यह TTL केश के पीछे लाइव प्रदाता API को क्वेरी करती है, और — शासन के अनुसार — पुराने हार्डकोडेड फॉलबैक स्तर को पूरी तरह हटा दिया गया था। जब लाइव खोज विफल होती है, तो LLMProvider कुछ भी वापस नहीं करता, बजाय एक पुराने कैटलॉग के, ताकि कॉलर को कभी ऐसा मॉडल आईडी न दिया जाए जो वैध दिखे लेकिन जिसे लागू न किया जा सके। इस सबका हर हिस्सा समवर्ती उपयोग के लिए थ्रेड-सुरक्षित बनाया गया है।

सामग्री

हमने इसे क्यों बनाया

नैव ज़ेड9टीईआरएम007ज़ेड9 कॉल प्रोडक्शन में विफल हो जाते हैं — प्रदाता दर-सीमित करते हैं, सेवा गिरा देते हैं या बंद हो जाते हैं, और एक खराब बैकएंड पूरे सेवा को अपने साथ ले डूबता है। मॉडल कैटलॉग बदलते रहते हैं, और हार्डकोडेड सूचियाँ ऐसे कॉलर आईडी थमा देती हैं जो अब काम नहीं करते। ज़ेड9टीईआरएम002ज़ेड9 इंटरफ़ेस को केंद्रीकृत करता है, लचीलापन पैटर्न और ईमानदार खोज को इस तरह से एकीकृत करता है कि हर उपभोक्ता को दोष सहनशीलता और सच्चाई मुफ्त में मिल जाती है।

यह गेम-चेंजर क्यों है

यह “ज़ेड9टीईआरएम007ज़ेड9 प्रदाता को एकीकृत करें” की प्रक्रिया को एक ही कदम में समेट देता है — एक इंटरफ़ेस लागू करें, या बस जेनेरिक एडाप्टर को किसी एंडपॉइंट पर निर्देशित करें — और फिर उस प्रदाता को स्वचालित और पारदर्शी रूप से सर्किट ब्रेकिंग, स्वास्थ्य निगरानी और ज़िटर-बैकऑफ़ रिट्राई में लपेट देता है। लचीलापन अब हर टीम के लिए अलग से आविष्कार की जाने वाली चीज़ नहीं रह जाती (खराब तरीके से, डेडलाइन के

दबाव में, पहले आउटेज के बाद), बल्कि यह लाइब्रेरी का डिफ़ॉल्ट व्यवहार बन जाता है, जो सभी 43 बैकएंड पर लागू होता है। विश्वसनीयता इंजीनियरिंग एक बार लिखी जाती है, कड़ी परीक्षा से गुजरती है, और इसे आयात करने वाले हर व्यक्ति को मुफ्त में मिल जाती है।

क्या है नवाचारी

- **एक क्षमता-सचेत इंटरफ़ेस** — पूरा होना, स्ट्रीमिंग, स्वास्थ्य, क्षमताएँ और कॉन्फ़िगरेशन सत्यापन एक ही अनुबंध में समाहित हैं, जिसे हर बैकएंड समान रूप से मानता है।
- **पारदर्शी सर्किट-ब्रेकर रैपिंग** — स्ट्रीम सहित। यह ब्रेकर `CompleteStream` के चैनल की रक्षा करता है, न कि केवल अनुरोध/प्रतिक्रिया की, और खाली स्ट्रीम को असफलता के रूप में मानता है — जिसमें डेडलॉक-सुरक्षित, ऑफ-लॉक श्रोता अधिसूचना शामिल है।
- **43 प्रदाता पैकेज + एक जेनेरिक जेड9टीईआरएम005जेड9-संगत एडाप्टर** — समर्पित पैकेज पतले रहते हैं, और `/v1/chat/completions` बोलने वाला कोई भी अपंजीकृत विक्रेता एडाप्टर को उस पर निर्देशित करते ही काम करने लगता है।
- **एकल क्रेडेंशियल प्राधिकरण (apikeys)** — केवल एक ही स्थान `ApiKey_<Provider>` पर्यावरण चर पढ़ता है, जिससे “हरे परीक्षण, टूटा उत्पाद” के बेमेल को संरचनात्मक रूप से समाप्त किया जाता है, न कि केवल चेतावनी देकर।
- **ईमानदार मॉडल खोज (हार्डकोडेड फ़ॉलबैक नहीं)** — TTL कैश के पीछे लाइव प्रदाता API; विफलता पर यह `nil` लौटाता है, कभी पुराना या गढ़ा हुआ कैटलॉग नहीं जो अनुपयोगी आईडी थमा दे।
- **आलसी आरंभीकरण `sync.Once` के साथ** — निर्माण का काम पहले उपयोग तक टाला जाता है, इसलिए सभी 43 प्रदाताओं को पंजीकृत करने की लागत तब तक लगभग शून्य रहती है जब तक आप वास्तव में किसी को कॉल नहीं करते।
- **धोखाधड़ी-रोधी, बहु-स्थानीय चुनौती स्टैक** — एक वास्तविक रनर जो पाँच स्थानों पर सर्किट, स्वास्थ्य और रिट्राई व्यवहार का परीक्षण करता है, जिसे युग्मित म्यूटेशन परीक्षण द्वारा नियंत्रित किया जाता है (अपरिवर्तित कोड को शून्य से बाहर निकलना चाहिए; इंजेक्ट किया गया म्यूटेशन 99 से बाहर निकलने को बाध्य करता है), ताकि पास होने वाला सूट सिद्ध रूप से काम करने वाले व्यवहार का प्रमाण दे।

सबसे बड़ी तकनीकी चुनौतियाँ और उनके समाधान

- **प्रदाताओं की श्रृंखलाबद्ध विफलताएँ**। एक अस्थिर बैकएंड पूरे सेवा को अपने साथ नहीं ले डूबना चाहिए। इसका समाधान तीन-अवस्था सर्किट ब्रेकर (बंद → खुला → अर्ध-खुला) से किया गया, जो किसी भी प्रदाता और उसकी स्ट्रीम को पारदर्शी रूप से लपेटता है, निरंतर विफलता पर खुल जाता है, अर्ध-खुली अवस्था में पुनर्प्राप्ति की जाँच करता है, और इसे `CircuitBreakerManager` द्वारा केंद्रीय रूप से समन्वित किया जाता है।
- **क्षणिक त्रुटियाँ और दर सीमाएँ**। इसका समाधान स्थिति-सचेत घातांकीय बैकऑफ़ और ज़िटर से किया गया — $\min(\text{InitialDelay} \cdot \text{Multiplier}^{(n-1)}, \text{MaxDelay}) \pm \text{ज़िटर}$ — ताकि रिट्राई फैल जाएँ और “थंडरिंग हर्ड” में समन्वित न हों। यह केवल उन्हीं स्थितियों में रिट्राई करता है जिनमें रिट्राई की जानी चाहिए (429, 500, 502, 503, 504, और नेटवर्क त्रुटियाँ), और रद्द किए गए संदर्भ या किसी अन्य 4xx पर प्रयास बर्बाद नहीं करता।
- **कई पंजीकृत लेकिन अप्रयुक्त प्रदाताओं का स्केलिंग**। 43 प्रदाता पंजीकृत होने पर भी किसी सेवा में केवल कुछ ही सक्रिय होते हैं, इसलिए उत्सुक निर्माण पूरी तरह व्यर्थ होता। इसका समाधान `sync.Once` द्वारा संरक्षित आलसी आरंभीकरण से किया गया,

ताकि केवल उन्हीं प्रदाताओं को उनकी स्थापना लागत चुकानी पड़े जिन्हें आप वास्तव में कॉल करते हैं।

- **अमान्य मॉडल आईडी थमा देना**। इसका समाधान हार्डकोडेड खोज फ़ॉलबैक स्तर को पूरी तरह हटाकर (CONST-036 के अनुसार) और लाइव-खोज विफलता पर कुछ भी नहीं लौटाकर किया गया — साथ ही रक्षात्मक प्रतिलिपि-लौटाने से ताकि कॉलर कैश को बदल न सके या दूसरे पाठक के साथ दौड़ न लगा सके। सच्चाई को संरचनात्मक रूप से लागू किया जाता है, केवल परंपरा से नहीं।
- **स्ट्रीमिंग + समवर्ती शुद्धता**। सूक्ष्म विफलता मोड ब्रेकर के लॉक और उसके श्रोता कॉलबैक के बीच डेडलाक है। इसका समाधान श्रोताओं का स्नैपशॉट लेकर और उन्हें ऑफ-लॉक अधिसूचित करके 5-सेकंड टाइमआउट के तहत किया गया, और रीसेट पर अधिसूचना से पहले अनलॉक करके — हर घटक समवर्ती उपयोग के लिए बनाया गया और -race सूट द्वारा सत्यापित।

सामग्री

तकनीकी स्टैक

- **Go (1.25.3)** — प्रथम श्रेणी की समवर्तीता (कॉनकरेंसी), स्थैतिक बाइनरी और मज़बूत मानक लाइब्रेरी के लिए चुना गया; इसमें मॉड्यूल, इंटरफ़ेस, हर लचीलापन प्रिमिटिव और सभी 43 एडेप्टर शामिल हैं।
- **net/http (मानक लाइब्रेरी)** — जानबूझकर निर्भरता-मुक्त HTTP: प्रति-प्रदाता क्लाइंट, सामान्य OpenAI-संगत एडेप्टर और लाइव डिस्कवरी कॉल को शक्ति प्रदान करता है, इसलिए किसी तीसरे पक्ष के ट्रांसपोर्ट की ऑडिट या पैचिंग की ज़रूरत नहीं।
- **logrus** — संरचित, स्तर-सचेत लॉगिंग, ठीक वहाँ जहाँ ऑपरेटरों को दृश्यता चाहिए: सर्किट ब्रेकर की स्थिति परिवर्तनों और डिस्कवरी पथ के भीतर।
- **testify** — टेस्ट सूट को संचालित करता है और, महत्वपूर्ण रूप से, म्यूटेशन-ब्रांच पिनिंग जो एक सफल रन को अर्थपूर्ण बनाती है।
- **yaml.v3** — i18n बंडल और कॉन्फ़िगरेशन को ऐसे प्रारूप में पार्स करता है जो मानव-संपादन योग्य बना रहता है।
- **digital.vasic.models** — साझा LLMRequest / LLMResponse / ProviderCapabilities प्रकार, एक ही स्थान पर रखे गए ताकि हर एडेप्टर एक ही शब्दावली का उपयोग करे (एक प्रलेखित रनटाइम निर्भरता)।
- **स्वयं के पैकेज** — circuit, health, retry, apikeys, discovery, providers/ (43 विक्रेता + generic), और i18n: लचीलापन और एकीकरण सतह को छोटी, स्वतंत्र रूप से परीक्षण योग्य इकाइयों में विभाजित किया गया है, न कि एक मोनोलिथ में।
- **.env + ~/api_keys.sh (ApiKey_<Provider> परंपरा)** — क्रेडेंशियल की एकल, स्पष्ट स्रोत, ताकि कुंजियाँ परीक्षण और उत्पादन में एक ही तरीके से संयोजित हों।
- **Makefile रेस सूट (-race -p 1) + चैलेंज रनर** — विरोधाभास-विरोधी आधारशिला: रेस डिटेक्टर समवर्तीता की शुद्धता साबित करता है, और चैलेंज रनर वास्तविक व्यवहार को अराजकता, डीडॉस, स्कलिंग, तनाव, लाइव-डिस्कवरी और निलंबन-रहित परिदृश्यों के माध्यम से परखता है।

स्थिति और ईमानदारी संबंधी टिप्पणियाँ

- **स्थिति:** बीटा। एक अलग पुनःप्रयोग योग्य मॉड्यूल; GitHub रिपॉजिटरी सार्वजनिक है।

- लाइसेंस: निर्धारित नहीं। असंगत — doc.go MIT कहता है जबकि Apache-2.0 शैली का LICENSE फ़ाइल मौजूद है — प्रकाशन से पहले सत्यापित करें।
- LLMsVerifier अपस्ट्रीम मॉडल कैटलॉग का एकमात्र स्रोत है। helix-deps.yaml मेनिफ़ेस्ट पुराना प्रतीत होता है (deps: [] घोषित करता है जबकि दस्तावेज़ digital.vasic.models पर निर्भरता बताते हैं); डिस्कवरी का “टियर 2 (models.dev)” एक नियोजित स्टब है, सक्रिय नहीं।

प्राथमिकता स्तर: Helix-प्राथमिक (LLM-इन्फ्रास्ट्रक्चर क्लस्टर — अलग पुनःप्रयोग योग्य मॉड्यूल)। HelixTrack के बाद आता है।